

ProofFund × Sourcify

How ProofFund turns Sourcify's verified contracts dataset into an AI-native reputation engine that gates access to grant funding.

ProofFund team May 2026 Sepolia testnet · open source

1 — The problem we're solving

Onchain grant programs and DAOs keep running into the same issue: capital goes to teams that pitch well, not necessarily to teams that build well. There's no shared, trustless way to check whether a developer actually has a track record of shipping and verifying smart contracts.

The core idea behind ProofFund: before any capital moves, an AI agent audits the applicant's deployer wallet using Sourcify's verified contracts dataset. Only developers who clear this bar can open a prediction-market-governed grant.

Sourcify is the only public dataset we know of that gives us the kind of evidence we need: not just "this contract exists", but *how carefully was it verified, documented, and structured?* That's the signal that separates serious builders from wallets that deployed one test contract three years ago.

2 — Where Sourcify sits in the system

ProofFund is built as a four-layer stack. Sourcify powers the very first layer — the AI reputation audit — which acts as the gate before anything else happens.

PROOFFUND STACK		
Layer 1	AI Reputation Audit	Gate: Sourcify-powered audit; must pass to proceed.
Layer 2	Funding Markets	Prediction markets decide which approved builders actually receive grants.
Layer 3	ENS Identity	AI keeps a live reputation record in ENS text fields.
Layer 4	KPI + Treasury	Onchain KPIs unlock treasury tranches programmatically.

Layer 1 is impossible without Sourcify. The other three layers build on top of that gate.

When a developer submits a grant application, the first thing the intake agent does is look up their deployer wallet in Sourcify's BigQuery dataset. No Sourcify data means no score. No score means the application doesn't proceed.

3 — What we actually query from Sourcify

We use three access methods — BigQuery (primary), the Sourcify API v2, and the 4byte Signature API — and we go well beyond surface-level calls.

BigQuery — deployer-to-contract join

The main query starts from the deployer wallet and follows the join chain that Sourcify's schema makes possible:

```
public_contract_deployments (deployer wallet → contract addresses, chain IDs)
  ↓ join on deployment_id
public_verified_contracts (match quality: creation, runtime, metadata flags)
  ↓ join on compilation_id
public_compiled_contracts (ABI, storageLayout, devdoc, userdoc, compiler version)
  ↓ join on source_hash
public_sources (raw Solidity/Vyper source code → security pattern scan)
```

This gives us a full picture of every contract a wallet has ever verified — not just "how many", but what quality, on which chains, how well documented, and whether the source contains any dangerous patterns.

Sourcify API v2 — per-contract enrichment

For each contract the BigQuery query returns, we call `GET /v2/contract/{chainId}/{address}?fields=all` to enrich with fields not easily joinable at scale: `proxyResolution.isProxy`, `verifiedAt`, and the structured ABI array for function/event/error counts.

4 — How the reputation score is computed

The audit produces a score from 0 to 1 — a weighted sum of six criteria, all derived from Sourcify data. Here's what each criterion measures and why it's weighted the way it is:

Criterion	Weight	Sourcify fields used	What it captures
Verification quality	0.25	<code>creation_match</code> , <code>runtime_match</code> , <code>runtime_metadata_match</code> , <code>creation_metadata_match</code>	How carefully the developer verified. A full metadata match is significantly harder to fake than a partial runtime match.
Documentation	0.20	<code>devdoc</code> , <code>userdoc</code> , <code>storageLayout</code> (from <code>compilation_artifacts</code>)	Engineering culture signal. Documented contracts are easier to audit, upgrade, and trust.
Has verified contracts	0.15	Row count from <code>public_verified_contracts</code> joined via <code>deployment_id</code>	Base gate: zero verified contracts is a near-automatic reject.
Activity history	0.15	<code>created_at</code> from <code>public_contract_deployments</code> (span + recency)	Consistent shipping over time. Recent activity matters.
Complexity + multichain	0.15	ABI function/event/error counts; <code>chain_id</code> from <code>public_contract_deployments</code>	Non-trivial systems and multi-L2 deployment signal real production work.
Security patterns	0.10	Source code text from <code>public_sources.content</code> (regex scan)	Flags <code>selfdestruct</code> , <code>tx.origin</code> , unprotected <code>emergencyWithdraw</code> , rug-pull patterns.

Chain weights are applied to production deployments (Ethereum mainnet = 1.0, Base/Optimism/Arbitrum = 0.9, testnets = 0.05–0.1). A proxy deployment bonus adds a small signal for developers comfortable with upgrade patterns.

Score → verdict → action

Score	Verdict	What happens next
≥ 0.65	APPROVE	Prediction market opens automatically

Score	Verdict	What happens next
0.35 – 0.64	REVIEW	Manual review queue, extra diligence
< 0.35	REJECT	Application closed, no market opened

5 — A real audit: Uniswap deployer

To show the system works on real data, we ran the audit on the Uniswap deployer wallet (`0x41653c7d...TQTR`). BigQuery returned 30 verified contracts. The Sourcify API enriched each one. Here's the output:

```
===== SOURCIFY RELIABILITY AUDIT Developer:
0x41653c7d61609D856f29355E404F310Ec4142Cfb =====
Score: 0.671 / 1.000 ✔ APPROVE Criterion Fill Got / Max
===== has_any_verified [████████████████████] 0.150
/ 0.15 ↳ 30 contracts (4 production) verification_quality [██████████░░░░░░░░░░] 0.142 / 0.25 ↳
Weighted match quality: 0.57 documentation [████░░░░░░░░░░░░░░░░░░░░] 0.046 / 0.20 ↳ Doc quality: 0.23
(devdoc/userdoc/storageLayout) activity_history [████████████████████] 0.138 / 0.15 ↳ 1.7yr span,
last ~0 months ago complexity [██████████░░░░░░░░░░░░░░░░░░░░] 0.095 / 0.15 ↳ Complexity: 0.49, prod chains:
[1, 8453] security [████████████████████] 0.100 / 0.10 ↳ Clean — no dangerous patterns Summary: • 30
verified contracts (4 on mainnet / L2) • Activity history: 1.7 years • Last verified ~0 months ago •
Weak documentation (common for Uniswap — mostly interface-only) • Clean code ✔ • Multi-chain
developer: chains [1, 8453] =====
```

The weak documentation score is interesting — Uniswap contracts are typically well-engineered but skip `devdoc/userdoc` since they rely on external specs. The score still passes at 0.671 because the other signals are strong. This is exactly the kind of nuance the system captures.

6 — Why this is deep use of Sourcify, not surface-level

The Sourcify bounty asks for projects that "exploit the unique properties of the data — source code, metadata, storage layouts, compiler settings" rather than surface-level API calls. Here's how ProofFund uses each layer:

Sourcify asset	How ProofFund uses it
<code>creation_match</code> / <code>runtime_match</code>	Primary quality signal. Weighted 0.25 in the score — the most impactful single criterion.
<code>runtime_metadata_match</code> / <code>creation_metadata_match</code>	Secondary match signal — metadata match is harder to achieve and signals more careful compilation.
<code>devdoc</code> / <code>userdoc</code>	Documentation quality score. Weighted 0.20 — signals engineering culture.
<code>storageLayout</code>	Storage layout presence signals upgrade-aware development and auditability.
ABI array (functions, events, errors)	Used to compute interface complexity. More non-trivial functions = higher complexity score.

Sourcify asset	How ProofFund uses it
Source code (<code>public_sources.content</code>)	Full regex scan for dangerous patterns: <code>selfdestruct</code> , <code>tx.origin</code> , <code>emergencyWithdraw</code> , rug-pull structures.
<code>deployer</code> from <code>public_contract_deployments</code>	The root of the entire query — every audit starts with a wallet address lookup here.
<code>chain_id</code> + block data	Chain weight and multi-chain bonus. Production L2s weighted higher than testnets.
<code>proxyResolution.isProxy</code> (Sourcify API)	Proxy detection bonus. Proxy patterns signal experience with upgradeable systems.

The key novelty: Sourcify has always been used to find and verify contracts. ProofFund uses it to verify *people*. The same dataset that tells you "is this contract source-available?" now tells you "is this developer trustworthy enough to receive a grant?" — a fundamentally new use of the data.

7 — What we don't use yet (and why)

We're being transparent about this because the bounty rewards honest evaluation.

Unused Sourcify asset	Why not in v1	Could add
Parquet file export	BigQuery covers the same data with easier querying for our use case	Useful for offline batch scoring
Deep source analysis (<code>public_compiled_contracts_sources</code>)	Full AST analysis is expensive; we do pattern detection as a proxy	LLM-powered source audit in v2
Compiler settings (optimizer runs, <code>viaIR</code> , <code>evmVersion</code>)	Useful for deep audits; adds noise at grant-screening level	Evasion heuristic for very old compiler versions
Function signatures (<code>public_signatures</code>)	Code already prepared but not wired into the score	Admin function detection (hidden owner controls)
Bytecode similarity (<code>public_code</code>)	Copy-paste detection is a v2 feature	Flag cloned/forked deployments

8 — Running the audit

Two requirements: a Google Cloud service account with **BigQuery Data Viewer** on the `whaleteam-495709` linked dataset, and the Python dependencies.

```
pip install google-cloud-bigquery requests

# Quick start — audit any deployer wallet:
from sourcify_audit import audit_developer

result = audit_developer(
    "0x41653c7d61609D856f29355E404F310Ec4142Cfb",
    max_contracts=30
)
print(result["score"])    # 0.671
print(result["verdict"])  # APPROVE
```

The function returns a structured dict: `score`, `verdict`, `breakdown` (per-criterion), and `summary` (human-readable strings for the agent to use in the prediction market description). The whole audit for one wallet runs in roughly 15–30 seconds depending on how many contracts the BigQuery query returns.

The module is open source, MIT-licensed, and will be in the team's public GitHub repo before the end of the hackathon. Live demo: `demo.prooffund.eth` on Sepolia.

9 — One sentence on why this fits the bounty

The Sourcify bounty is looking for creative, deep, useful, and novel use of the verified contracts dataset. ProofFund uses Sourcify as the trust anchor for an entirely new kind of system — one where verified contracts aren't just a way to read code, but a way to underwrite the humans who wrote it.